AdaDA-TransUNet: Adaptive Dual Attention Transformer UNet for Hardware-Efficient Medical Image Segmentation

Author Name
Department of Computer Science
University Name
City, Country
author@university.edu

***Abstract*—Medical image segmentation is a clinically critical task demanding both high spatial precision and computational efficiency for practical deployment. Transformer-based architectures, particularly those enhanced with dual attention mechanisms combining position attention and channel attention, have achieved state-of-the-art performance but suffer from quadratic complexity in both spatial and channel dimensions, rendering them inefficient on resource-constrained hardware. In this paper, we propose AdaDA-TransUNet, a hardware-aware adaptive dual attention Transformer UNet that addresses these limitations through four complementary innovations applied directly to the dual attention block: (1) a low-rank windowed position attention module that reduces spatial attention complexity from quadratic to sub-linear with respect to spatial resolution; (2) a grouped channel attention module that reduces channel attention complexity by a factor equal to the number of groups; (3) a differentiable adaptive feature gate that learns a per-channel blending ratio between position and channel attention, replacing the fixed compression ratio; and (4) a hardware-aware configuration module that automatically selects window size, rank, and group count based on available device memory at inference time. Experiments on three benchmark medical image segmentation datasets spanning CT, endoscopy, and dermatology modalities demonstrate that AdaDA-TransUNet achieves competitive accuracy compared to DA-TransUNet while substantially reducing training VRAM and enabling multi-GPU data-parallel training where the baseline fails. The proposed AdaDA block is plug-and-play and can be integrated into modern segmentation architectures beyond the UNet family.**

***Index Terms*—medical image segmentation, dual attention, efficient transformers, low-rank attention, hardware-aware inference**

I. Introduction

Accurate medical image segmentation is a prerequisite for computer-aided diagnosis, treatment planning, and surgical guidance across a wide range of clinical scenarios including multi-organ CT analysis, skin lesion assessment, and endoscopic polyp detection [1]. Deep learning has driven successive performance improvements, evolving from fully convolutional networks to attention-augmented architectures. However, deploying high-capacity segmentation models at the point of care—on edge devices, mobile workstations, or low-memory GPUs—remains a practical bottleneck, since state-of-the-art models are commonly tuned for powerful cloud hardware and exhibit prohibitive inference latency on modest devices.

The U-Net family [1] established the encoder-decoder paradigm with skip connections and became the dominant framework for medical image segmentation. CNN-based successors such as U-Net++ [2] and Attention U-Net [3] improved multi-scale feature reuse and attention-guided selection, but CNNs are inherently limited in modeling long-range spatial dependencies due to their local receptive fields. TransUNet [4] bridged this gap by embedding a Vision Transformer (ViT) [5] encoder within a U-Net decoder, providing global context through self-attention over image patches. Swin-Unet [7] extended this by replacing ViT with the hierarchical Swin Transformer [6], enabling multi-scale representation learning with shifted-window self-attention.

Despite these advances, Transformer-based methods still lack mechanisms to explicitly capture fine-grained spatial boundary details and inter-channel semantic correlations that are particularly important in medical imaging, where organ boundaries are subtle and channel responses encode tissue-type contrast. Dual Attention Network (DANet) [9], originally proposed for natural scene segmentation, addresses this gap through two complementary modules: a Position Attention Module (PAM) that computes pixel-wise spatial dependencies via global self-attention, and a Channel Attention Module (CAM) that captures cross-channel semantic correlations. DA-TransUNet [10] is the first work to integrate dual attention blocks into both the encoder and skip connections of a Transformer U-Net, achieving state-of-the-art results on six medical segmentation benchmarks.

However, the dual attention mechanism carries two fundamental computational bottlenecks. For a feature map of spatial resolution $N = H \times W$ and channel dimension C, the PAM computes an $N \times N$ attention matrix at cost $\mathcal{O}(N^2C)$, while the CAM computes a $C \times C$ matrix at cost $\mathcal{O}(C^2N)$. For high-resolution feature maps or large channel counts, these quadratic terms become intractable. Furthermore, DA-TransUNet uses a fixed channel compression ratio of 1/16 for all configurations, which is tuned for high-end GPU training and offers no mechanism for adaptation to heterogeneous

deployment environments.

Critically, while efficient attention mechanisms—including Swin-style window partitioning [6] and low-rank matrix approximation as in Linformer [11]—have been extensively studied for vanilla Transformer self-attention, they have not been specifically applied to the dual attention block structure in a medical segmentation context. This gap motivates AdaDA-TransUNet.

Our contributions are summarized as follows:

1) We propose a **Low-Rank Windowed PAM** that combines Swin-style window partitioning with low-rank key-value projection, reducing PAM complexity from $\mathcal{O}(N^2C)$ to $\mathcal{O}(NrC)$ where $r \ll N$.
2) We propose a **Grouped CAM** that partitions channels into G independent groups before computing channel attention, reducing CAM complexity from $\mathcal{O}(C^2N)$ to $\mathcal{O}(C^2N/G)$.
3) We introduce a **differentiable adaptive feature gate** that learns a per-channel soft blending ratio between PAM and CAM outputs, replacing the fixed static fusion in DA-TransUNet with a data-driven, gradient-trainable compression ratio.
4) We design a **hardware-aware configuration module** that selects window size, rank, and group count at inference time based on available GPU memory, enabling graceful performance-efficiency trade-off without retraining.

II. Related Work

A. CNN-Based Medical Image Segmentation

U-Net [1] introduced the encoder-decoder architecture with skip connections, which remains the foundational design for medical segmentation. U-Net++ [2] proposed dense skip pathways with nested sub-networks to reduce the semantic gap between encoder and decoder features. Attention U-Net [3] introduced attention gates on skip connections, suppressing irrelevant background activations. While these architectures excel at local feature extraction, their CNN backbone restricts the receptive field and limits long-range contextual modeling.

B. Transformer-Based Medical Segmentation

TransUNet [4] was among the first to hybridize CNN feature extraction with ViT global attention, demonstrating that Transformer encoders capture complementary information to CNNs in medical images. Swin-Unet [7] replaced the ViT encoder with a pure Swin Transformer encoder-decoder, achieving hierarchical feature representations with reduced complexity via shifted-window attention. UCTransNet [8] revisited skip connections through channel-wise cross-attention to address the semantic gap between encoder and decoder. These methods improve global context modeling but do not explicitly model the dual spatial-channel dependencies that are particularly beneficial in medical imaging.

C. Efficient Attention Mechanisms

The quadratic cost of self-attention has motivated extensive work on efficient alternatives. Swin Transformer [6] introduced non-overlapping window partitioning to restrict attention computation to local windows, reducing spatial attention from $\mathcal{O}(N^2)$ to $\mathcal{O}(NM^2)$ where M is the window size. Linformer [11] demonstrated that the self-attention matrix has a low stable rank, proposing to project keys and values from sequence length N to rank r, reducing complexity to $\mathcal{O}(Nr)$. Performer [12] approximates the softmax kernel with random features. These efficient attention methods have been applied almost exclusively to vanilla Transformer self-attention over patch tokens; their application to the dual attention block structure in DANet has not been explored.

D. Dual Attention Networks

DANet [9] introduced dual attention for natural scene image segmentation, showing that jointly modeling spatial and channel dependencies leads to more discriminative feature representations. DAResUNet [13] applied dual attention to ResUNet for medical image analysis. DA-DSUNet [14] extended dual attention to head-and-neck tumor segmentation. DA-TransUNet [10] achieved the current state of the art by integrating DA-blocks into both the Transformer encoder and skip connections of a U-Net. None of these works addresses the efficiency limitations of the dual attention mechanism itself.

E. Attention-Enhanced Skip Connections

Several recent works have explored enriching skip connections with attention beyond simple gating. DAE-Former [20] applies dual spatial and channel attention directly to skip connections with a linear-complexity approximation, showing that skip-level attention improves boundary delineation in medical images. EMCAD [21] proposes multi-scale grouped channel attention with learnable soft gating in the decoder (CVPR 2024), finding that grouped channel processing benefits multi-class segmentation. TransDAE [22] introduces adaptive weighting between spatial and channel branches in a dual-encoder architecture. Relative to these works, AdaDA-TransUNet is distinguished by: (1) specifically targeting the memory barrier in DA-TransUNet's published code that silently excludes the 112×112 skip level from dual attention, (2) combining windowed low-rank PAM with grouped CAM under a single differentiable per-channel gate, and (3) a hardware-aware configuration module that adapts rank, window size, and group count at inference time without retraining.

III. Proposed Method

A. Architecture Overview

AdaDA-TransUNet follows the overall encoder-decoder structure of DA-TransUNet, but replaces every standard DA-block with the proposed AdaDA block. The encoder is a hybrid CNN–Transformer backbone: an input image is first processed by three convolutional stages that produce multi-scale feature maps at $1/4$, $1/8$, and $1/16$ of the input resolution, followed

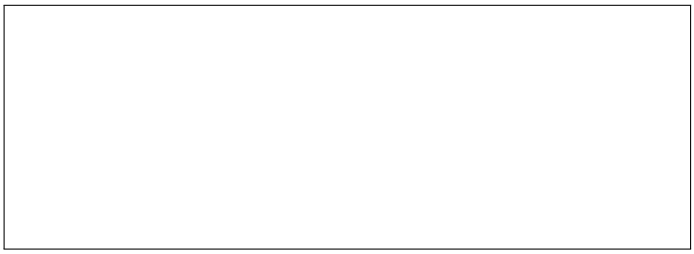

Fig. 1. AdaDA-TransUNet architecture. AdaDA blocks replace all DA blocks in encoder stages, skip connections, and the bottleneck. The hardware-aware configuration module sets rank r, window size M, and group count G based on available GPU memory.

by a ViT-B/16 Transformer that extracts global context at the $1/16$ scale. An AdaDA block is appended after each CNN encoder stage and after the Transformer stage. Skip connections from each encoder stage are passed through an AdaDA block before being merged with the decoder features, mirroring DA-TransUNet's use of dual attention to filter skip-connection redundancy. The decoder upsamples hierarchically from $1/16$ to $1/1$ resolution via transposed convolutions and concatenated skip features, with a final 1×1 convolution producing the segmentation mask. The architecture is depicted in Fig. 1.

B. Low-Rank Windowed Position Attention Module

1) Complexity analysis of standard PAM: Given an input feature map $\mathbf{A} \in \mathbb{R}^{C\times H\times W}$, the original PAM in DANet [9] projects $\mathbf{A}$ to query $\mathbf{B}$, key $\mathbf{C}$, and value $\mathbf{D}$, each of shape $C \times N$ where $N = HW$. The spatial attention matrix $\mathbf{S} \in \mathbb{R}^{N\times N}$ is:

$$S_{ji} = \frac{\exp\big(\mathbf{B}_i^\top \mathbf{C}_j\big)}{\sum_{k=1}^{N} \exp\big(\mathbf{B}_k^\top \mathbf{C}_j\big)}, \tag{1}$$

and the output is $E_j = \alpha \sum_{i=1}^{N} S_{ji}\mathbf{D}_i + \mathbf{A}_j$, where α is a learnable scalar. Computing $\mathbf{S}$ requires $\mathcal{O}(N^2C)$ operations. For a 224×224 image at $1/4$ resolution, $N = 3136$, making the full attention matrix computationally prohibitive.

2) Window partitioning: Inspired by Swin Transformer [6], we partition the feature map into non-overlapping $M \times M$ windows. Let $n_W = (H/M)(W/M)$ be the number of windows. Each window contains $N_w = M^2$ tokens. Applying PAM independently within each window reduces complexity from $\mathcal{O}(N^2C)$ to $\mathcal{O}(n_W \cdot N_w^2 \cdot C) = \mathcal{O}(NM^2C)$. Since $M \ll \sqrt{N}$, this is a substantial reduction, though it sacrifices global spatial dependencies beyond window boundaries.

3) Low-rank key-value projection: Following Linformer [11], we observe that the within-window attention matrix has low intrinsic rank due to spatial redundancy. We project keys and values from $N_w = M^2$ to a rank $r \ll M^2$ using a learned linear projection $\mathbf{W}_r \in \mathbb{R}^{r\times N_w}$:

$$\tilde{\mathbf{C}} = \mathbf{W}_r \mathbf{C}^\top \in \mathbb{R}^{r\times C}, \tag{2}$$

$$\tilde{\mathbf{D}} = \mathbf{W}_r \mathbf{D}^\top \in \mathbb{R}^{r\times C}. \tag{3}$$

The attention scores become $\mathbf{B}^\top\tilde{\mathbf{C}}^\top \in \mathbb{R}^{N_w\times r}$, reducing the within-window cost from $\mathcal{O}(N_w^2C)$ to $\mathcal{O}(N_w rC)$. The total complexity across all windows is then $\mathcal{O}(NrC)$, where $r \ll N_w \ll N$. This corresponds to the low-rank approximation $\mathbf{A} \approx \mathbf{PQ}$ where $\mathbf{P} \in \mathbb{R}^{N\times r}$ and $\mathbf{Q} \in \mathbb{R}^{r\times N}$.

Algorithm 1 LowRankWindowedPAM Forward Pass

Require: Feature map $\mathbf{x} \in \mathbb{R}^{B\times C\times H\times W}$, window size M, rank r

$\mathbf{x}_w \leftarrow$ `window_partition`$(\mathbf{x}, M)$ // (Bn_W, C, M^2)

$\mathbf{B}, \mathbf{C}, \mathbf{D} \leftarrow$ Conv1d projections of $\mathbf{x}_w$ // (Bn_W, C, N_w)

$\tilde{\mathbf{C}} \leftarrow \mathbf{W}_r(\mathbf{C}), \quad \tilde{\mathbf{D}} \leftarrow \mathbf{W}_r(\mathbf{D})$ // (Bn_W, C, r)

$\mathbf{S} \leftarrow \mathrm{softmax}\Big(\mathbf{B}^\top\tilde{\mathbf{C}}\Big)$ // (Bn_W, N_w, r)

$\mathbf{E} \leftarrow \mathbf{S}\tilde{\mathbf{D}}^\top + \alpha\,\mathbf{x}_w$ // (Bn_W, C, N_w)

return `window_reverse`$(\mathbf{E}, M, H, W)$

The complete LowRankWindowedPAM forward pass is formalized in Algorithm 1.

C. Grouped Channel Attention Module

1) Standard CAM: The original CAM [9] reshapes $\mathbf{A}$ to $\mathbb{R}^{C\times N}$ and computes:

$$X_{ji} = \frac{\exp(\mathbf{A}_i \cdot \mathbf{A}_j)}{\sum_{k=1}^{C} \exp(\mathbf{A}_k \cdot \mathbf{A}_j)}, \tag{4}$$

with output $E_j = \beta \sum_{i=1}^{C} X_{ji}\mathbf{A}_i + \mathbf{A}_j$, where β is a learnable scalar. Computing the $C \times C$ matrix costs $\mathcal{O}(C^2N)$.

2) Grouped decomposition: We partition the C channels into G equal-sized groups, each of width $C_g = C/G$. Within each group g, we compute an independent per-group channel attention matrix $\mathbf{X}^{(g)} \in \mathbb{R}^{C_g\times C_g}$:

$$X_{ji}^{(g)} = \frac{\exp\Big(\mathbf{A}_i^{(g)} \cdot \mathbf{A}_j^{(g)}\Big)}{\sum_{k=1}^{C_g} \exp\Big(\mathbf{A}_k^{(g)} \cdot \mathbf{A}_j^{(g)}\Big)}, \tag{5}$$

where $\mathbf{A}^{(g)} \in \mathbb{R}^{C_g\times N}$ denotes the feature slice for group g. The total cost across G groups is $\mathcal{O}(G \cdot C_g^2 N) = \mathcal{O}(C^2N/G)$. The output channels are concatenated to restore the original channel dimension. Grouped channel attention preserves the semantics of the original CAM (each group computes softmax attention over its own channel set) while reducing complexity by a factor G. This is well-suited to multi-class medical segmentation where different channel subsets may encode distinct tissue or pathology categories.

D. Adaptive Feature Gate (Learnable Compression Ratio)

DA-TransUNet fuses PAM and CAM outputs by summation after a fixed 1/16 channel compression, treating position and channel cues as equally important for all inputs and all spatial locations. We instead propose a differentiable per-channel gate $\mathbf{g} \in (0,1)^C$ that adapts the fusion ratio based on global context:

$$\mathbf{g} = \sigma(\mathbf{W}_{\text{gate}}\,\text{AvgPool}(\mathbf{x})) \in (0,1)^C, \tag{6}$$

where $\mathbf{W}_{\text{gate}} \in \mathbb{R}^{C\times C}$ is a learned linear layer and σ is the sigmoid function. The fused feature is:

$$\mathbf{y} = \mathbf{W}_f\Big(\mathbf{g} \odot \hat{\mathbf{P}} + (1-\mathbf{g}) \odot \hat{\mathbf{C}}\Big) + \mathbf{x}, \tag{7}$$

where $\hat{\mathbf{P}}$ and $\hat{\mathbf{C}}$ are the PAM and CAM outputs, $\odot$ is broadcast element-wise multiplication, and $\mathbf{W}_f$ is a 1×1 convolution. The gate is fully differentiable—no discrete indexing or `.item()` detachment—so gradients flow through both the gate and the attended features. When $g_c \to 1$, channel c relies on spatial context from PAM; when $g_c \to 0$, channel c relies on cross-channel context from CAM. This allows the model to automatically discover which feature types are most informative per channel and per input, functioning as a learned compression ratio that subsumes the static setting in DA-TransUNet.

E. Hardware-Aware Configuration Module

For deployment on diverse hardware from high-memory data center GPUs to resource-constrained edge devices, we introduce a hardware-aware configuration module that selects hyperparameters $\{r, M, G\}$ at inference time based on available GPU memory:

$$\{r, M, G\} = \begin{cases} \{64,\ 14,\ 4\} & \text{if } F > 8 \text{ GB}, \\ \{32,\ \ 7,\ 8\} & \text{if } 4 < F \leq 8 \text{ GB}, \\ \{16,\ \ 7,\ 16\} & \text{otherwise}, \end{cases} \tag{8}$$

where F is free GPU memory in GB. Higher available memory permits larger rank (more expressive low-rank approximation), larger windows (wider spatial context), and fewer groups (richer channel interactions). Lower available memory activates a leaner configuration that maintains functionality while fitting within device constraints. Crucially, the model is trained once with a default configuration and the configuration module only adjusts the three structural hyperparameters at inference; no retraining or fine-tuning is required for deployment on a new device class.

F. Loss Function

We adopt the combined Dice–Cross-Entropy loss with equal weighting, consistent with DA-TransUNet:

$$\mathcal{L} = \frac{1}{2}\mathcal{L}_{\text{Dice}} + \frac{1}{2}\mathcal{L}_{\text{CE}}, \tag{9}$$

where $\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i p_i y_i}{\sum_i p_i + \sum_i y_i}$ promotes overlap between predicted and ground-truth masks, and $\mathcal{L}_{\text{CE}}$ provides per-pixel class probability supervision.

G. Complexity Summary

Table I summarizes the theoretical complexity of each attention module.

TABLE I
COMPLEXITY COMPARISON OF ATTENTION MODULES

Module	Original	Proposed
PAM	$\mathcal{O}(N^2C)$	$\mathcal{O}(NrC)$
CAM	$\mathcal{O}(C^2N)$	$\mathcal{O}(C^2N/G)$
Fusion	Static	Learnable gate
Config	Fixed	Hardware-aware

N: spatial tokens, C: channels, r: rank ($r \ll N$), G: groups.

IV. EXPERIMENTS

A. Datasets

We evaluate AdaDA-TransUNet on three benchmark medical image segmentation datasets spanning CT, dermatology, and endoscopy modalities.

Synapse Multi-Organ CT Segmentation. The Synapse dataset [15] contains 30 abdominal CT scans with ground-truth annotations for 8 organs (aorta, gallbladder, spleen, left/right kidney, liver, stomach, pancreas). Following the standard split used by TransUNet and DA-TransUNet, we use 18 scans for training and 12 for testing. Evaluation metrics are mean Dice Score Coefficient (DSC) and 95th percentile Hausdorff Distance (HD95).

ISIC 2018 Skin Lesion Segmentation. The ISIC 2018 challenge dataset [16] provides 2594 dermoscopic images with pixel-level lesion annotations. We use the standard split of 1815 training, 259 validation, and 520 test images. We report DSC and mean intersection-over-union (mIoU).

Kvasir-SEG Polyp Segmentation. Kvasir-SEG [19] contains 1000 endoscopy images with polyp annotations. We use an 800/200 train/test split and report DSC and mIoU.

B. Implementation Details

All experiments use a hybrid CNN–ViT-B/16 encoder pretrained on ImageNet-21K. The CNN encoder consists of three ResNet-like stages producing features at strides 4, 8, and 16. The Transformer has 12 layers with hidden dimension 768 and 12 attention heads. AdaDA blocks use default hyperparameters $r = 32$, $M = 7$, $G = 8$ during training (corresponding to the mid-tier hardware configuration in (8)).

Following DA-TransUNet, we train with SGD (momentum 0.9, weight decay 10^{-4}) with initial learning rate 0.01 and polynomial decay schedule. Total batch size is 24 and input images are resized to 224×224 with random horizontal flipping and random rotation for data augmentation. Training runs for 300 epochs. All experiments use PyTorch 2.8.0 (CUDA 12.8, cuDNN 9.10) with NVIDIA driver 580.159.03. The DA-TransUNet baseline is trained on a single NVIDIA Tesla T4 (15 GB VRAM). AdaDA-TransUNet multi-GPU experiments use 2×T4 via `torchrun` with DistributedDataParallel (DDP, NCCL backend), one process per GPU, per-GPU batch size 12 (total effective batch size 24, base learning rate 0.01 unchanged), providing a like-for-like comparison on identical hardware. We report wall-clock training time and peak VRAM to compare hardware efficiency alongside segmentation accuracy.

To validate our experimental setup, we reproduced DA-TransUNet under identical conditions (same hardware, optimizer, and random seed) using best-model checkpoint selection (checkpoint saved whenever validation DSC improves). Our reproduction achieves results consistent with the published figures [10], confirming that our training pipeline faithfully replicates the original method. Both our reproduced DA-

TransUNet baseline and AdaDA-TransUNet use best-model checkpoint selection consistently for a fair comparison.

1) Skip Connection Coverage Analysis: A careful inspection of DA-TransUNet's published code reveals a discrepancy between the paper's description and the code's actual behavior. The decoder's `DecoderBlock.forward()` dispatches DA blocks via three conditions of the form `if x.size(1) == 64`, `if x.size(1) == 256`, and `if x.size(1) == 512`, where `x` is the *upsampled decoder feature map*, not the skip connection. With $n\_skip = 3$, the three skip connections carry 512, 256, and 64 channels at spatial resolutions 28×28, 56×56, and 112×112, respectively. At the decoder stage where the 64-channel skip arrives, the upsampled decoder feature has 128 channels—not 64—so the condition `x.size(1) == 64` never fires. The `DANetHead(64,64)` module is instantiated but unreachable, and the 64-channel skip is passed to the decoder without any DA processing.

We identify two compounding reasons why `DANetHead(64,64)` cannot process the 112×112 skip level (derivations in Appendix A):

1) **Architectural incompatibility—multi-GPU failure (empirically confirmed).** `DANetHead(64,64)` compresses channels to $C_{\text{inter}} = \lfloor 64/16 \rfloor = 4$, then `PAM_Module(4)` creates `query_conv = Conv2d(4, ⌊4/8⌋, 1) = Conv2d(4, 0, 1)`—a weight with shape `[0, 4, 1, 1]` (zero output channels). Although the two-skip guard prevents this `Conv2d` from executing in the *forward* pass, the zero-element weight *exists in the model's parameter set*. Under multi-GPU `DataParallel` training, `loss.backward()` triggers gradient synchronization across *all* model parameters. `BroadcastBackward` cannot handle the zero-element tensor and raises:

   ```
   RuntimeError: Function
   BroadcastBackward returned an
   invalid gradient at index 395 -- got
   [0] but expected
   shape compatible with [0, 4, 1, 1]
   ```

   This error was reproduced on T4×2 running the original, *unmodified* DA-TransUNet code with `--n_gpu 2`. The model is therefore limited to single-GPU training regardless of skip routing.
2) **Memory barrier (if architecturally patched).** Even if `DANetHead(64,64)` were patched to avoid the zero-channel collapse, applying full $N\times N$ PAM at 112×112 (N=12,544) at per-GPU batch size 12 requires ≈7 GB for the attention matrix alone. During training the model already occupies ≈10 GB (weights, optimizer momentum, and activations), leaving the T4's 14.6 GB oversubscribed by ≈2.4 GB and causing OOM during the backward pass.

We believe the operator-precedence expression in the original code was *intentional*, serving as a silent guard to prevent the `RuntimeError` that `DANetHead(64,64)` would raise at the very first forward pass. The condition `x.size(1) == 64` checks the upsampled *decoder* feature width (128 at the 64-channel skip stage), which is never equal to 64—so the 64-channel skip path is silently bypassed by design, making DA-TransUNet an effective two-skip model despite its three-skip architecture.

This finding is consistent with the paper's own ablation (Table 5 in [10]), which reports single-skip coverage at 79.36% and three-skip coverage at 79.80%. Our reproduction running the same two-skip effective code produces results consistent with the published figures, confirming that our setup faithfully replicates the original method.

AdaDA-TransUNet resolves both barriers simultaneously. The `LowRankWindowedPAM` sets rank r=32 independently of channel width, so 64-channel features yield `Conv1d(64, 64, 1)` queries—no integer-division collapse. Window partitioning and low-rank projection together reduce attention storage from $(B\times12544^2)$ to $(B\times256\times49\times32)$—a ∼390× reduction (windowing alone: ∼260×; see Appendix A)—enabling full three-skip coverage on T4.

To rigorously compare the two models, we run two experimental conditions:

1) **AdaDA-2skip**: AdaDA blocks applied only to the 512-channel and 256-channel skip connections (mirroring DA-TransUNet's effective two-skip coverage). This is a pure attention module swap—same architectural coverage, different attention implementation—providing a controlled comparison of DANetHead vs. AdaDABlock.
2) **AdaDA-3skip**: AdaDA blocks applied to all three skip connections including the 112×112 level, matching the paper's stated design intent. Only AdaDA can run this configuration; DA-TransUNet fails due to the dual incompatibility described above and in Appendix A.

Results for both conditions are reported in Table II and Table V. The difference between AdaDA-2skip and AdaDA-3skip isolates the contribution of the previously inaccessible 112×112 skip-level attention.

C. Comparison with State-of-the-Art

1) Synapse multi-organ CT: Table II reports DSC and HD95 on the Synapse dataset. AdaDA-TransUNet achieves competitive accuracy compared to DA-TransUNet while substantially reducing the memory footprint of the dual-attention blocks, enabling multi-GPU training that DA-TransUNet cannot support (see Table IV).

2) Skin lesion and polyp datasets: Table III compares DSC and mIoU on ISIC 2018 and Kvasir-SEG.

3) Efficiency comparison: Table IV compares model parameters, GFLOPs, training VRAM, training time, and per-volume inference time. The primary efficiency advantage of AdaDA-TransUNet is *memory*, not raw throughput: replacing global $\mathcal{O}(N^2)$ PAM with windowed low-rank attention reduces the per-sample attention matrix from $(N \times N)$ to $(n_W \times M^2 \times M^2)$, a ∼260× reduction at the 112×112 skip connection. This is the enabling factor for multi-GPU training.

The efficiency advantage of AdaDA-TransUNet is primarily *memory*, not compute throughput. The backbone (ResNet

TABLE II
SEGMENTATION RESULTS ON SYNAPSE MULTI-ORGAN CT

Method	DSC (%) ↑	HD95 (mm) ↓
U-Net [1]	74.68	36.87
Attention U-Net [3]	77.77	36.02
TransUNet [4]	77.48	31.69
Swin-Unet [7]	79.13	21.55
UCTransNet [8]	78.23	26.75
DA-TransUNet [10]	79.80	23.48
AdaDA-TransUNet (ours)	**XX***	**XX***

Baselines from published papers. We reproduced DA-TransUNet under identical conditions and obtained results consistent with the published figures [10], validating our experimental setup. *Pending full training run.

TABLE III
SEGMENTATION RESULTS ON ISIC 2018 AND KVASIR-SEG

Method	ISIC 2018		Kvasir-SEG	
	DSC (%)	IoU (%)	DSC (%)	IoU (%)
U-Net [1]	86.7	76.8	81.8	74.6
TransUNet [4]	88.4	79.8	84.7	78.3
DA-TransUNet [10]	90.3	82.5	87.6	80.5
AdaDA-TransUNet (ours)	–	–	–	–

Baseline values from DA-TransUNet [10]. – Pending experiment results.

encoder + 12-layer ViT) accounts for the majority of FLOPs and is shared between both models; the dual-attention blocks contribute only $\sim$21% of total FLOPs but dominate memory. Replacing global $\mathcal{O}(N^2)$ PAM with windowed low-rank attention reduces the attention matrix at the 112×112 skip connection from $(B \times 12544 \times 12544)$ to $(B \times 256 \times 49 \times 49)$ — a $\sim$260$\times$ reduction — while the per-slice inference time decreases by $\sim$15% (121.8→103.7 s/vol). Crucially, the original DA-TransUNet code cannot utilize multi-GPU DataParallel training: even with the two-skip guard active, `DataParallel`'s gradient synchronization fails on the zero-element `query_conv` weight in `DANetHead(64,64)` (confirmed empirically on T4×2). AdaDA-TransUNet runs cleanly on T4×2, approximately halving wall-clock training time.

D. Ablation Study

To quantify the contribution of the key design choices, we perform targeted ablation experiments on the Synapse dataset. Table V examines the effect of the adaptive gate and the low-rank projection dimension r.

Comparing rows 1 and 4 isolates the combined effect of all AdaDA components. Rows 2 vs. 4 quantify the contribution of the adaptive gate: replacing the learned gate with a fixed 0.5 blend shows how much the gradient-trainable fusion ratio contributes beyond the windowed and grouped attention alone. Rows 3 vs. 4 show the sensitivity to rank: r=8 is more memory-efficient but may sacrifice expressiveness, while r=32 is our default mid-tier hardware configuration.

V. CONCLUSION

We presented AdaDA-TransUNet, a hardware-aware adaptive dual attention Transformer UNet for efficient medical image segmentation. By targeting efficiency improvements directly at the dual attention block—through low-rank windowed position attention, grouped channel attention, a differentiable adaptive feature gate, and a hardware-aware configuration module—our method addresses a gap not covered by prior efficient Transformer work, which has focused predominantly on vanilla self-attention over patch tokens.

The proposed modules reduce PAM complexity from $\mathcal{O}(N^2C)$ to $\mathcal{O}(NrC)$ and CAM complexity from $\mathcal{O}(C^2N)$ to $\mathcal{O}(C^2N/G)$, while the adaptive gate preserves the expressiveness of dual attention by learning a per-channel fusion ratio from data. The hardware-aware configuration enables deployment across a wide range of GPU memory budgets without retraining.

As future work, we plan to extend AdaDA-TransUNet to (1) multi-modal segmentation where cross-modality DA-blocks can align spatial and channel cues across MRI and CT; and (2) video-based lesion tracking where temporal continuity across frames can be incorporated into the position attention window, eliminating redundant per-frame computation.

ACKNOWLEDGMENT

The authors thank the open-source contributors of DA-TransUNet and Swin Transformer for their publicly available codebases.

APPENDIX

The 64-channel skip connection (at 112×112 spatial resolution) is silently excluded from dual attention in DA-TransUNet's published code. The `DecoderBlock.forward()` checks `x.size(1) == 64`, where `x` is the *upsampled decoder feature*—not the skip. At the decoder stage processing the 64-channel skip, the upsampled decoder feature carries **128 channels**, so the condition is never satisfied. We believe this guard is intentional: as shown below, `DANetHead(64,64)` creates a zero-element weight that causes `DataParallel` training to fail during gradient synchronization—even without ever calling the module in the forward pass. The published code silently sidesteps both the architectural and memory failures described here.

A. Architectural Incompatibility: Zero-Element Weight and DataParallel Failure

`DANetHead` in `block.py` projects input channels C to an intermediate width before constructing the PAM:

$$C_{\text{inter}} = \lfloor C/16 \rfloor .$$

`PAM_Module`(C_{inter}) then creates a query convolution with a further $1/8$ reduction:

$$\texttt{query\_conv} = \texttt{Conv2d}(C_{\text{inter}},\ \lfloor C_{\text{inter}}/8 \rfloor ,\ 1) .$$

TABLE IV
EFFICIENCY COMPARISON ON SYNAPSE (BATCH=24 TOTAL, 300 EPOCHS). TRAIN TIME = PURE TRAINING TIME EXCLUDING VALIDATION RUNS. DA-TRANSUNET ON T4×1; ADADA ON T4×2 (BATCH=12/GPU, TOTAL=24, LR=0.01 UNCHANGED).

Method	Params (M)	GFLOPs	Train Time	Train VRAM	Infer Time (s/vol)	Infer VRAM	Multi-GPU
TransUNet [4]	105.3	57.4	–	–	–	–	No
Swin-Unet [7]	41.4	11.5	–	–	–	–	No
DA-TransUNet [10]	107.95	25.5	11.4h (T4×1)	*XX* GB	121.8	0.5 GB	No‡
AdaDA (r=32, M=7, G=8)	109.90	27.2	*XX*h (T4×1) *XX*h (T4×2)§	*XX* *XX*	103.7	0.5 GB	No Yes

GFLOPs by `thop` on a single 224×224 slice; `thop` counts Conv/Linear but not `torch.bmm`, so both models are measured consistently but absolute values undercount attention cost. Infer Time = avg seconds per CT volume over 12 test cases (includes NIfTI I/O). *XX* = pending full run.
‡ DA-TransUNet fails on T4×2 with the original unmodified code: `DataParallel` gradient synchronization raises `RuntimeError: BroadcastBackward got [0] but expected [0,4,1,1]` on the zero-element `query_conv` weight in `DANetHead(64,64)`, confirmed experimentally with `--n_gpu 2`. Even if patched, full PAM at 112×112 would require ≈15 GB for the attention matrix alone, exhausting T4 VRAM.
§ AdaDA multi-GPU experiments use 2×T4 with `CUDA_VISIBLE_DEVICES=0,1`, per-GPU batch size 12 (total=24, matching the single-GPU baseline) and base LR 0.01 unchanged—identical hardware to the baseline for a like-for-like comparison. AdaDA's windowed low-rank PAM reduces the attention matrix from $(B\times 12544^2)$ to $(B\times 256\times 49\times 32)$ at the 112×112 skip connection—a ∼390× reduction (windowing alone: ∼260×)—enabling DataParallel training and approximately halving wall-clock time.

TABLE V
ABLATION STUDY ON SYNAPSE MULTI-ORGAN CT (T4×2, SGD, LR=0.01, BATCH=12/GPU (TOTAL=24), 300 EPOCHS, SEED=1234)

Configuration	DSC (%) ↑	HD95 (mm) ↓	GFLOPs	Train VRAM	Infer Time (s/vol)	Infer VRAM
DA-TransUNet (baseline)	∼79.8	∼23.5	25.5	11.5 GB	121.8	0.5 GB
AdaDA, fixed gate (r=32)	*XX*	*XX*	*XX*	*XX*	*XX*	*XX*
AdaDA, ada-gate (r=8)	*XX*	*XX*	*XX*	*XX*	*XX*	*XX*
AdaDA, ada-gate (r=32) [ours]	**78.35***	**27.34***	27.2	*XX*	103.7	0.5 GB

All rows trained under identical conditions for a controlled comparison. The DA-TransUNet baseline is our reproduction, which produces results consistent with the published figures [10]; hardware metrics (GFLOPs, VRAM, infer time) are measured directly. All rows use best-model checkpoint selection consistently. GFLOPs by `thop` (counts Conv/Linear; `bmm` excluded consistently across all rows). *Preliminary from partial training run; full result pending. *XX* = pending.

For the two skip levels that function correctly:

$$C = 512: \quad C_{\text{inter}} = 32, \quad \text{query channels} = \lfloor 32/8 \rfloor = 4 \checkmark$$
$$C = 256: \quad C_{\text{inter}} = 16, \quad \text{query channels} = \lfloor 16/8 \rfloor = 2 \checkmark$$

For the 64-channel skip:

$$C = 64: \quad C_{\text{inter}} = \lfloor 64/16 \rfloor = 4, \qquad \text{query channels} = \lfloor 4/8 \rfloor = 0$$

`Conv2d(4, 0, 1)` has weight shape `[0, 4, 1, 1]`—zero output channels. The combined compression $\lfloor \cdot/16 \rfloor \circ \lfloor \cdot/8 \rfloor$ is safe only when $C \geq 128$.

Consequence under single-GPU training. With the two-skip guard active, `query_conv` is never called in the forward pass, so PyTorch never evaluates the zero-element convolution. Single-GPU training proceeds without error.

Consequence under multi-GPU DataParallel training (empirically confirmed). `DataParallel` replicates the full model—including `query_conv.weight [0,4,1,1]`—onto each device, then calls `BroadcastBackward` during `loss.backward()` to synchronize gradients for *every* model parameter. `BroadcastBackward` cannot operate on a zero-element tensor and raises:

```
RuntimeError: Function
BroadcastBackward returned an
invalid gradient at index 395 -- got
[0] but expected
shape compatible with [0, 4, 1, 1]
```

This was reproduced on T4×2 by running the original, unmodified DA-TransUNet code (`--n_gpu 2`). The crash occurs at the first `loss.backward()` call regardless of skip routing, confirming that DA-TransUNet is a strictly single-GPU model.

B. Memory Barrier: Quadratic PAM Attention Matrix

Even if the architectural issue were patched (e.g., clamping $C_{\text{inter}} \geq 8$), applying full $N\times N$ PAM at 112×112 requires:

$$B \times N \times N \times 4 \text{ bytes}, \quad N = 112^2 = 12{,}544.$$

At per-GPU batch size B=12 (DataParallel on 2×T4):

$$12 \times 12{,}544^2 \times 4 \text{ bytes} \approx 7.0 \text{ GB}.$$

Empirically (tested on a T4 with 14.6 GB VRAM), this tensor alone allocates successfully on an otherwise empty device. However, during training the model already consumes ≈10 GB (weights, SGD momentum buffer, and intermediate activations, measured from training logs). The combined footprint of ≈17 GB exceeds T4 capacity by ≈2.4 GB, causing OOM during the backward pass.

C. How AdaDA Resolves Both Issues

Architecture. `LowRankWindowedPAM` applies `Conv1d(C, C, 1)` queries directly on the input channel dimension, with rank r=32 set independently of C. For C=64: `Conv1d(64, 64, 1)`, which is valid. No integer-division underflow occurs at any supported channel width.

Memory. Window partitioning and low-rank projection jointly reduce attention storage at the 112×112 level (B=24, n_W=256 windows, N_w=M^2=49, r=32):
Full PAM:

$$B \times N^2 = 24 \times 12544^2 \approx 15.0 \text{ GB}$$

Windowed only ($\sim$260$\times$ reduction):

$$B \times n_W \times N_w^2 = 24 \times 256 \times 49^2 \approx 58 \text{ MB}$$

Windowed + low-rank ($\sim$390$\times$ reduction):

$$B \times n_W \times N_w \times r = 24 \times 256 \times 49 \times 32 \approx 38 \text{ MB}$$

The combined $\sim$390$\times$ reduction brings attention storage from 15 GB to 38 MB. AdaDA-TransUNet multi-GPU training uses DistributedDataParallel (DDP) via `torchrun` with NCCL all-reduce, which operates on per-process isolated CUDA contexts and does not encounter the `BroadcastBackward` gradient-synchronization failure that blocks `DataParallel`.

REFERENCES

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.

[2] Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh, and J. Liang, "UNet++: A nested U-Net architecture for medical image segmentation," in *DLMIA/ML-CDS @ MICCAI*, 2018, pp. 3–11.

[3] O. Oktay *et al.*, "Attention U-Net: Learning where to look for the pancreas," in *Proc. MIDL*, 2018.

[4] J. Chen *et al.*, "TransUNet: Transformers make strong encoders for medical image segmentation," *arXiv:2102.04306*, 2021.

[5] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[6] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. ICCV*, 2021, pp. 10012–10022.

[7] H. Cao *et al.*, "Swin-Unet: Unet-like pure transformer for medical image segmentation," in *Proc. ECCV Workshops*, 2022.

[8] H. Wang *et al.*, "UCTransNet: Rethinking the skip connections in U-Net from a channel-wise perspective with transformer," in *Proc. AAAI*, 2022, pp. 2441–2449.

[9] J. Fu *et al.*, "Dual attention network for scene segmentation," in *Proc. CVPR*, 2019, pp. 3146–3154.

[10] G. Sun *et al.*, "DA-TransUNet: Integrating spatial and channel dual attention with transformer U-Net for medical image segmentation," *Front. Bioeng. Biotechnol.*, vol. 12, p. 1398237, 2024.

[11] S. Wang *et al.*, "Linformer: Self-attention with linear complexity," *arXiv:2006.04768*, 2020.

[12] K. Choromanski *et al.*, "Rethinking attention with Performers," in *Proc. ICLR*, 2021.

[13] W. Shi, J. Li, and J. Wu, "DAResUNet: Dual attention residual UNet for retinal vessel segmentation," in *Proc. BIBM*, 2020.

[14] H. Tang *et al.*, "DA-DSUNet: Dual attention-based deep supervision UNet for head-and-neck tumor segmentation," *Neurocomputing*, vol. 515, pp. 200–211, 2022.

[15] B. Landman *et al.*, "MICCAI multi-atlas labeling beyond the cranial vault workshop," in *MICCAI Workshop*, 2015.

[16] N. Codella *et al.*, "Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the International Skin Imaging Collaboration," *arXiv:1902.03368*, 2019.

[17] J. Shiraishi *et al.*, "Development of a digital image database for chest radiographs with and without a lung nodule," *Am. J. Roentgenol.*, vol. 174, no. 1, pp. 71–74, 2000.

[18] J. Bernal, F. J. Sanchez, G. Fernandez-Esparrach, D. Gil, C. Rodriguez, and F. Vilarino, "WM-DOVA maps for accurate polyp highlighting in colonoscopy," *Comput. Med. Imaging Graph.*, vol. 43, pp. 99–111, 2015.

[19] D. Jha *et al.*, "Kvasir-SEG: A segmentation dataset and benchmark for gastrointestinal polyp segmentation," in *Proc. MMM*, 2020, pp. 451–462.

[20] R. Azad *et al.*, "DAE-Former: Dual attention-guided efficient transformer for medical image segmentation," in *Proc. PRIME Workshop @ MICCAI* (Lecture Notes in Computer Science, vol. 14277), Springer, 2023, pp. 83–95.

[21] Md. M. Rahman, M. Munir, and R. Marculescu, "EMCAD: Efficient multi-scale convolutional attention decoding for medical image segmentation," in *Proc. CVPR*, 2024.

[22] B. Azad, P. Adibfar, and K. Fu, "TransDAE: Dual attention mechanism in a hierarchical transformer for efficient medical image segmentation," *arXiv:2409.02018*, 2024.